`rollingInterval` parameter. Your log file will be written to a folder within the application named Logs. You don't need to create the folder yourself because Serilog will create it if it doesn't exist.

The existing content in Program.cs is placed in a `try-catch` block, so any exceptions raised during the startup phase are caught and logged. Listing 12.19 shows the code from the `CreateBuilder` method call all the way down to the final `app.Run` call being placed in a `try` block, with the addition of the `UseSerilog` method call, which removes existing logging providers and enables Serilog as their replacement

Listing 12.19 Handling and logging any potential startup exceptions

```
try                                                          ❶
{
    var builder = WebApplication.CreateBuilder(args);
    builder.Host.UseSerilog();                               ❷
    builder.Services.AddRazorPages(options =>
    ...
    app.Run();
}
catch (Exception ex)                                         ❸
{
    Log.Fatal(ex, "Application start failed");
}
finally                                                      ❹
{
    Log.Information("Shut down complete");
    Log.CloseAndFlush();
}
```

❶ Place a try block around the existing code in Program.cs.

❷ Add a call to UseSerilog on the Host property of the WebApplicationBuilder.

❸ Log any exceptions encountered during app configuration and start.

❹ Flush any buffered log messages to their destination.

Serilog log levels are mostly identical to those provided by the .NET logging system, except the lowest level is called `Verbose` (instead of `Trace`),